

Contraste de momentum para aprendizagem de representação visual não supervisionada

Kaiming He Haoqi Fan Yuxin Wu Saining Xie Ross Girshick

Facebook AI Research (FAIR)

Código: <https://github.com/facebookresearch/moco>

Resumo

Apresentamos o Momentum Contrast (MoCo) para a aprendizagem de representação visual não supervisionada. A partir de uma perspectiva da aprendizagem contrastiva [29] como pesquisa em dicionário, construímos um dicionário dinâmico com uma fila e um codificador de média móvel. Isso permite a construção de um dicionário grande e consistente em tempo real, o que facilita a aprendizagem contrastiva não supervisionada. O MoCo fornece resultados competitivos sob o protocolo linear comum na classificação ImageNet. Mais importante ainda, as representações aprendidas pelo MoCo transferem-se bem para tarefas a jusante. O MoCo pode **superar** o seu equivalente de pré-treino supervisionado em 7 tarefas de detecção/segmentação no PASCAL VOC, COCO e outros conjuntos de dados, por vezes ultrapassando-o por margens significativas. Isto sugere que a diferença entre a aprendizagem de representação não supervisionada e supervisionada foi amplamente colmatada em muitas tarefas de visão.

1. Introdução

A aprendizagem de representação não supervisionada é altamente bem-sucedida no processamento de linguagem natural, por exemplo, conforme demonstrado pelo GPT [50, 51] e pelo BERT [12]. Mas o pré-treino supervisionado ainda é dominante na visão computacional, onde os métodos não supervisionados geralmente ficam para trás. A razão pode decorrer das diferenças nos seus respectivos espaços de sinal. As tarefas linguísticas têm espaços de sinal discretos (palavras, unidades sub-palavras, etc.) para a construção de dicionários tokenizados, nos quais a aprendizagem não supervisionada pode se basear. A visão computacional, por outro lado, preocupa-se ainda mais com a construção de dicionários [54, 9, 5], pois o sinal bruto está em um espaço contínuo e de alta dimensão e não é estruturado para a comunicação humana (por exemplo, ao contrário das palavras).

Vários estudos recentes [61, 46, 36, 66, 35, 56, 2] apresentam resultados promissores na aprendizagem de representação visual não supervisionada utilizando abordagens relacionadas com a perda contrastiva [29]. Embora motivados por várias razões, estes métodos podem ser considerados como a construção de dicionários dinâmicos. As «chaves» (tokens) no dicionário são amostradas a partir de dados (por exemplo, imagens ou patches) e são representadas por uma rede codificadora. A aprendizagem não supervisionada treina codificadores para realizar pesquisas no dicionário: uma «consulta» codificada deve ser semelhante à sua chave correspondente e diferente das outras. A aprendizagem é formulada como a minimização de uma perda contrastiva [29].

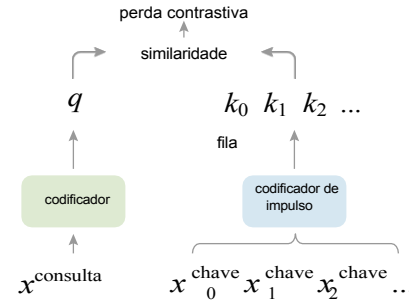


Figura 1. O Contraste de Momentum (MoCo) da treina um codificador de representação visual, combinando uma consulta codificada q com um dicionário de chaves codificadas, utilizando uma perda contrastiva. As chaves do dicionário $\{k_0, k_1, k_2, \dots\}$ são definidos dinamicamente por um conjunto de amostras de dados. O dicionário é construído como uma fila, com o mini-lote atual enfileirado e o mini-lote mais antigo retirado da fila, desacoplando-o do tamanho do mini-lote. As chaves são codificadas por um codificador de progressão lenta, impulsionado por uma atualização de momentum com o codificador de consulta. Este método permite um dicionário grande e consistente para a aprendizagem de representações visuais.

A partir dessa perspectiva, nossa hipótese é que é desejável construir dicionários que sejam: (i) grandes e (ii) consistentes à medida que evoluem durante o treinamento. Intuitivamente, um dicionário maior pode amostrar melhor o espaço visual contínuo e de alta dimensão subjacente, enquanto as chaves no dicionário devem ser representadas pelo mesmo codificador ou por um codificador semelhante, para que as suas comparações com a consulta sejam consistentes. No entanto, os métodos existentes que utilizam perdas contrastivas podem ser limitados num destes dois aspetos (discutido mais adiante no contexto).

Apresentamos o Momentum Contrast (MoCo) como uma forma de construir dicionários grandes e consistentes para aprendizagem não supervisionada com uma perda contrastiva (Figura 1). Mantemos o dicionário como uma fila de amostras de dados: as representações codificadas do mini-lote atual são enfileiradas e as mais antigas são retiradas da fila. A fila desacopla o tamanho do dicionário do tamanho do minilote, permitindo que ele seja grande. Além disso, como as chaves do dicionário vêm dos vários minilotes anteriores, um codificador de chaves de progressão lenta, implementado como uma média móvel baseada em momentum do codificador de consulta, é proposto para manter a consistência.

MoCo é um mecanismo para construir dicionários dinâmicos para aprendizagem contrastiva e pode ser usado com várias tarefas de pretexto. Neste artigo, seguimos uma tarefa simples de discriminação de instâncias [61, 63, 2]: uma consulta corresponde a uma chave se forem vistas codificadas (por exemplo, recortes diferentes) da mesma imagem. Usando essa tarefa de pretexto, o MoCo mostra resultados competitivos sob o protocolo comum de classificação linear no conjunto de dados ImageNet [11].

Um dos principais objetivos da aprendizagem não supervisionada é pré-treinar representações (ou seja, características) que podem ser transferidas para tarefas a jusante por meio de ajustes finos. Mostramos que, em 7 tarefas a jusante relacionadas à detecção ou segmentação, o pré-treinamento não supervisionado do MoCo pode *superar* o seu equivalente supervisionado do ImageNet, em alguns casos por margens não triviais. Nestas experiências, exploramos o MoCo pré-treinado no ImageNet ou num conjunto de *mil milhões* de imagens do Instagram, demonstrando que o MoCo pode funcionar bem num cenário mais realista, com mil milhões de imagens e relativamente sem curadoria. Esses resultados mostram que o MoCo diminui significativamente a diferença entre o aprendizado de representação não supervisionado e supervisionado em muitas tarefas de visão computacional e pode servir como uma alternativa ao pré-treinamento supervisionado do ImageNet em várias aplicações.

2. Trabalhos relacionados

Os métodos de aprendizagem não supervisionados/auto-supervisionados¹ geralmente envolvem dois aspetos: tarefas pretextuais e funções de perda. O termo «pretextual» implica que a tarefa a ser resolvida não é de interesse genuíno, mas é resolvida apenas com o verdadeiro objetivo de aprender uma boa representação de dados. As funções de perda podem frequentemente ser investigadas independentemente das tarefas pretextuais. O MoCo concentra-se no aspeto da função de perda. A seguir, discutimos estudos relacionados com esses dois aspetos.

Funções de perda. Uma maneira comum de definir uma função de perda é medir a diferença entre a previsão de um modelo e um alvo *fixo*, como reconstruir os pixels de entrada (por exemplo, autoencoders) por perdas L1 ou L2, ou classificar a entrada em categorias pré-definidas (por exemplo, oito posições [13], caixas de cores [64]) por perdas baseadas em margem ou entropia cruzada.

Outras alternativas, conforme descrito a seguir, também são possíveis.

As perdas contrastivas [29] medem as semelhanças entre pares de amostras num espaço de representação. Em vez de corresponder uma entrada a um alvo fixo, nas formulações de perda contrastiva o alvo pode *variar* dinamicamente durante o treino e pode ser definido em termos da representação de dados calculada por uma rede [29]. A aprendizagem contrastiva está no centro de vários trabalhos recentes sobre aprendizagem não supervisionada [61, 46, 36, 66, 35, 56, 2], que abordaremos mais adiante no contexto (Sec. 3.1).

As perdas adversárias [24] medem a diferença entre distribuições de probabilidade. É uma técnica amplamente bem-sucedida

para geração de dados não supervisionados. Métodos adversários para aprendizagem de representação são explorados em [15, 16]. Existem relações (ver [24]) entre redes adversárias generativas e estimativa contrastiva de ruído (NCE) [28].

Tarefas de pretexto. Uma ampla gama de tarefas de pretexto foi proposta. Exemplos incluem recuperar a entrada sob alguma corrupção, por exemplo, autoencoders de redução de ruído [58], autoencoders de contexto [48] ou autoencoders entre canais (colorização) [64, 65]. Algumas tarefas de pretexto formam pseudo-rótulos por meio, por exemplo, de transformações de uma única imagem (“exemplar”) [17], ordenação de patches [13, 45], rastreamento [59] ou segmentação de objetos [47] em vídeos, ou agrupamento de características [3, 4].

em vídeos ou agrupando características [3, 4].

Aprendizagem contrastiva vs. tarefas pretextuais. Várias tarefas pretextuais podem basear-se em alguma forma de funções de perda contrastiva. O método de discriminação de instâncias [61] está relacionado com a tarefa baseada em exemplares [17] e NCE [28]. A tarefa pretexto na codificação preditiva contrastiva (CPC) [46] é uma forma de auto-codificação de contexto [48] e, na codificação multiview contrastiva (CMC) [56], está relacionada com a colorização [64].

3. Método

3.1. Aprendizagem contrastiva como pesquisa em dicionário

A aprendizagem contrastiva [29] e os seus desenvolvimentos recentes podem ser considerados como o treino de um codificador para uma tarefa de *pesquisa em dicionário*, conforme descrito a seguir.

Considere uma consulta codificada q e um conjunto de amostras codificadas $\{k_0, k_1, k_2, \dots\}$ que são as chaves de um dicionário. Suponha que exista uma única chave (denotada como k_+) no dicionário.

dicionário que q corresponde. Uma perda contrastiva [29] é uma função cujo valor é baixo quando q é semelhante à sua chave positiva k_+ e diferente de todas as outras chaves (consideradas chaves negativas para q). Com a semelhança medida pelo produto escalar, uma forma de função de perda contrastiva, chamada InfoNCE [46], é considerada neste artigo:

$$L_q = - \log \frac{\exp(q \cdot k_+ / \tau)}{\sum_{i=0}^K \exp(q \cdot k_i / \tau)} \quad (1)$$

onde τ é um hiperparâmetro de temperatura por [61]. A soma é superior a uma amostra positiva e K amostras negativas. Intuitivamente, esta perda é a perda logarítmica de um classificador baseado em softmax de $(K+1)$ vias que tenta classificar q como k_+ . As funções de perda contrastiva também podem basear-se noutras formas [29, 59, 61, 36], tais como perdas baseadas em margens e variantes das perdas NCE.

A perda contrastiva serve como uma função objetiva não supervisionada para treinar as redes codificadoras que representam as consultas e as chaves [29]. Em geral, a representação da consulta é $q = f_q(x^q)$, onde f_q é uma rede codificadora e x^q é uma amostra de consulta (da mesma forma, $k = f_k(x^k)$). As suas instanciações dependem da tarefa específica do pretexto. As entradas x^q e x^k podem ser imagens [29, 61, 63], patches [46] ou contexto que consiste num conjunto de patches [46]. As redes f_q e f_k podem ser idênticas [29, 59, 63], parcialmente partilhadas [46, 36, 2] ou diferentes [56].

¹ A aprendizagem auto-supervisionada é uma forma de aprendizagem não supervisionada. A sua distinção é informal na literatura existente. Neste artigo, usamos o termo mais clássico de «aprendizagem não supervisionada», no sentido de «não supervisionada por rótulos anotados por humanos».

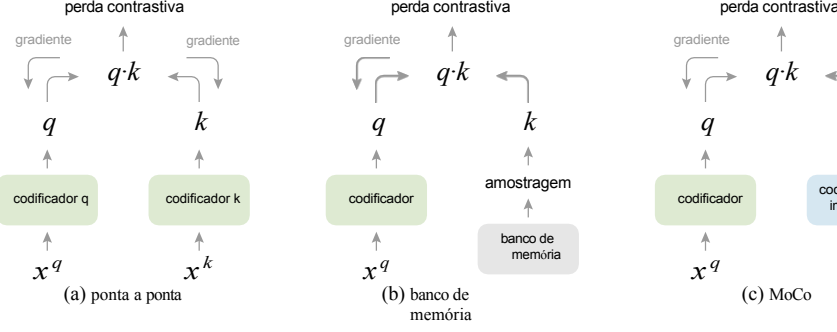


Figura 2. **Comparação conceitual de três mecanismos de perda contrastiva** (as comparações empíricas encontram-se na Figura 3 e na Tabela 3). Aqui ilustramos um par de consulta e chave. Os três mecanismos diferem na forma como as chaves são mantidas e como o codificador de chaves é atualizado. (a): Os codificadores para calcular as representações da consulta e da chave são atualizados *de ponta a ponta* por retropropagação (os dois codificadores podem ser diferentes). (b): As representações das chaves são amostradas a partir de um *banco de memória* [61]. (c): **O MoCo** codifica as novas chaves em tempo real por um codificador atualizado por momentum e mantém uma fila (não ilustrada nesta figura) de chaves.

3.2. Contraste de momentum

A partir da perspectiva acima, a aprendizagem contrastiva é uma forma de construir um dicionário discreto sobre entradas contínuas de alta dimensão, como imagens. O dicionário é *dinâmico* no sentido de que as chaves são amostradas aleatoriamente e que o codificador de chaves evolui durante o treino. A nossa hipótese é que boas características podem ser aprendidas por um *grande* dicionário que abrange um rico conjunto de amostras negativas, enquanto o codificador para as chaves do dicionário é mantido o mais *consistente* possível, apesar da sua evolução. Com base nessa motivação, apresentamos o Momentum Contrast, conforme descrito a seguir.

Dicionário como fila. No centro da nossa abordagem está a manutenção do dicionário como uma *fila* de amostras de dados. Isso permite-nos reutilizar as chaves codificadas dos mini-lotes imediatamente anteriores. A introdução de uma fila separa o tamanho do dicionário do tamanho do mini-lote. O tamanho do nosso dicionário pode ser muito maior do que o tamanho típico de um minilote e pode ser definido de forma flexível e independente como um hiperparâmetro. As amostras no dicionário são substituídas progressivamente.

O mini-lote atual é enfileirado no dicionário e o mini-lote mais antigo na fila é removido. O dicionário representa sempre um subconjunto amostrado de todos os dados, enquanto o cálculo extra de manter este dicionário é gerenciável. Além disso, remover o mini-lote mais antigo pode ser benéfico, porque as suas chaves codificadas são as mais desatualizadas e, portanto, as menos consistentes com as mais recentes.

Atualização do momentum. O uso de uma fila pode tornar o dicionário grande, mas também torna difícil atualizar o codificador de chaves por retropropagação (o gradiente deve propagar-se para todas as amostras na fila). Uma solução ingênua é copiar o codificador de chave f_k do codificador de consulta f_q , ignorando esse gradiente. Mas essa solução produz resultados ruins em experimentos (Seção 4.1). Nossa hipótese é que essa falha é causada pela rápida mudança do codificador, que reduz a consistência das representações das chaves. Propomos uma atualização do momentum para resolver esse problema.

Formalmente, denotando os parâmetros de f_k como θ_k e os de f_q como θ_q , atualizamos θ_k por:

$$\theta_k \leftarrow m\theta_k + (1 - m)\theta_q. \quad (2)$$

Aqui, $m \in [0, 1)$ é um coeficiente de momentum. Apenas os parâmetros θ_q são atualizados por retropropagação. O momentum

A atualização θ_k na Eq. (2) faz com que θ_k evolua mais suavemente do que θ_q . Como resultado, embora as chaves na fila sejam codificadas por codificadores diferentes (em mini-lotes diferentes), a diferença entre esses codificadores pode ser reduzida. Em experiências, um momentum relativamente grande (por exemplo, $m = 0,999$, o nosso padrão) funciona muito melhor do que um valor menor (por exemplo, $m = 0,9$), sugerindo que um codificador de chaves de evolução lenta é fundamental para fazer uso de uma fila.

Relações com mecanismos anteriores. O MoCo é um mecanismo geral para usar perdas contrastivas. Comparamo-lo com dois mecanismos gerais existentes na Figura 2. Eles exibem propriedades diferentes no tamanho e na consistência do dicionário.

A atualização *de ponta a ponta* por retropropagação é um mecanismo natural (por exemplo, [29, 46, 36, 63, 2, 35], Figura 2a). Ela usa amostras no minilote atual como dicionário, de modo que as chaves são codificadas de forma consistente (pelo mesmo conjunto de parâmetros do codificador). Mas o tamanho do dicionário está associado ao tamanho do minilote, limitado pelo tamanho da memória da GPU. Também é desafiado pela otimização de minilotes grandes [25]. Alguns métodos recentes [46, 36, 2] baseiam-se em tarefas pretextadas impulsionadas por posições locais, onde o tamanho do dicionário pode ser aumentado por múltiplas posições. Mas estas tarefas pretextadas podem exigir designs de rede especiais, tais como patchificar a entrada [46] ou personalizar o tamanho do campo recetivo [2], o que pode complicar a transferência destas redes para tarefas a jusante.

Outro mecanismo é a abordagem *do banco de memória* proposta por [61] (Figura 2b). Um banco de memória consiste nas representações de todas as amostras no conjunto de dados. O dicionário para cada minilote é amostrado aleatoriamente a partir do banco de memória sem retropropagação, de modo que pode suportar um grande tamanho de dicionário. No entanto, a representação de uma amostra em

Algoritmo 1 Pseudocódigo do MoCo em um estilo semelhante ao PyTorch.

```
# f_q, f_k: redes codificadoras para consulta e chave #
fila: dicionário como uma fila de K chaves (CxK) # m:
momentum
# t: temperatura

f_k.params = f_q.params # inicializar
para x no carregador: # carregar um minilote x com N amostras
    x_q = aug(x) # uma versão aumentada aleatoriamente
    x_k = aug(x) # outra versão aumentada aleatoriamente

    q = f_q.forward(x_q) # consultas: NxK
    k = f_k.forward(x_k) # chaves: NxK
    k = k.detach() # sem gradiente para chaves

    # logits positivos: Nx1
    l_pos = bmm(q.view(N,1,C), k.view(N,C,1))

    # logits negativos: NxK
    l_neg = mm(q.view(N,C), queue.view(C,K))

    # logits: Nx(1+K)
    logits = cat([l_pos, l_neg], dim=1)

    # perda contrastiva, Eqn. (1)
    labels = zeros(N) # positivos são a perda 0 =
    CrossEntropyLoss(logits/t, labels)

    # atualização SGD: consultar network
    loss.backward() update(f_q.params)

    # atualização do momentum: rede principal
    f_k.params = m * f_k.params + (1-m) * f_q.params

    # atualizar dicionário
    enqueue(fila, k) # enfileirar o minilote atual dequeue(fila) #
    retirar o minilote mais antigo
```

bmm: multiplicação de matrizes em lote; mm: multiplicação de matrizes; cat: concatenação.

O banco de memória foi atualizado quando foi visto pela última vez, portanto, as chaves amostradas são essencialmente sobre os codificadores em várias etapas diferentes ao longo do último período e, portanto, são menos consistentes. Uma atualização de momentum é adotada no banco de memória em [61]. Sua atualização de momentum é feita nas representações da mesma amostra, *não* no codificador. Essa atualização de momentum é irrelevante para o nosso método, porque o MoCo não rastreia todas as amostras. Além disso, nosso método é mais eficiente em termos de memória e pode ser treinado em dados na escala de bilhões, o que pode ser intratável para um banco de memória.

A Sec. 4 compara empiricamente estes três mecanismos.

3.3. Tarefa de pretexto

A aprendizagem contrastiva pode impulsionar uma variedade de tarefas de pretexto. Como o foco deste artigo não é a concepção de uma nova tarefa de pretexto, utilizamos uma tarefa simples, seguindo principalmente a tarefa de *discriminação de instâncias* em [61], à qual alguns trabalhos recentes [63, 2] estão relacionados.

Seguindo [61], consideramos uma consulta e uma chave como um par positivo se elas se originarem da mesma imagem e, caso contrário, como um par de amostras negativas. Seguindo [63, 2], tomamos duas «visualizações» aleatórias da mesma imagem sob aumento aleatório de dados para formar um par positivo. As consultas e as chaves são codificadas respetivamente pelos seus codificadores, f_q e f_k . O codificador pode ser qualquer rede neural convolucional [39].

O Algoritmo 1 fornece o pseudocódigo do MoCo para esta

tarefa pretexto. Para o mini-lote atual, codificamos as consultas e as suas chaves correspondentes, que formam os pares de amostras positivas. As amostras negativas são da fila.

Detalhes técnicos. Adotamos um ResNet [33] como codificador, cuja última camada totalmente conectada (após pooling médio global) tem uma saída de dimensão fixa (128-D [61]). Este vetor de saída é normalizado pela sua norma L2 [61]. Esta é a representação da consulta ou chave. A temperatura τ na Eq. (1) é definida como 0,07 [61]. A configuração de aumento de dados segue [61]: um recorte de 224×224 pixels é obtido a partir de uma imagem redimensionada aleatoriamente e, em seguida, é submetido a distorção aleatória de cor, inversão horizontal aleatória e conversão aleatória para escala de cinza, todos disponíveis no pacote torchvision do PyTorch.

BN aleatório. Os nossos f_q e f_k têm Batch codificadores f_q

Normalização (BN) [37] como no ResNet padrão [33]. Em experiências, descobrimos que o uso de BN impede o modelo de aprender boas representações, conforme relatado de forma semelhante em [35] (que evita o uso de BN). O modelo parece “enganar” a tarefa pretexto e encontra facilmente uma solução de baixa perda. Isso possivelmente ocorre porque a comunicação intra-lote entre as amostras (causada por BN) vazia informações.

Resolvemos esse problema embaralhando o BN. Treinamos com várias GPUs e executamos o BN nas amostras independentemente para cada GPU (como é feito na prática comum). Para o codificador de chave f_k , embaralhamos a ordem das amostras no mini-lote atual antes de distribuí-las entre as GPUs (e embaralhamos novamente após a codificação); a ordem das amostras do mini-lote para o codificador de consulta f_q não é alterada. Isso garante que as estatísticas do lote usadas para calcular uma consulta e sua chave positiva venham de dois subconjuntos diferentes. Isso resolve efetivamente o problema da trapaça e permite que o treinamento se beneficie do BN.

Usamos BN embaralhado tanto no nosso método quanto na sua contraparte de ablação ponta a ponta (Figura 2a). Isso é irrelevante para a contraparte do banco de memória (Figura 2b), que não sofre com esse problema porque as chaves positivas são de mini-lotes diferentes no passado.

4. Experiências

Estudamos o treinamento não supervisionado realizado em:

ImageNet-1M (IN-1M): Este é o conjunto de treino ImageNet [11] que tem cerca de 1,28 milhões de imagens em 1000 classes (frequentemente chamado ImageNet-1K; contamos o número de imagens, uma vez que as classes não são exploradas pela aprendizagem não supervisionada). Este conjunto de dados é bem equilibrado na sua distribuição de classes e as suas imagens geralmente contêm vistas icônicas de objetos.

Instagram-1B (IG-1B): Seguindo [44], este é um conjunto de dados de *aproximadamente 1 bilhão* (940 milhões) de imagens públicas do *Instagram*. As imagens são provenientes de cerca de 1500 hashtags [44] relacionadas às categorias do ImageNet. Este conjunto de dados é relativamente *menos organizado* em comparação com o IN-1M e tem uma distribuição *desequilibrada* e de *cauda longa* de dados do mundo real. Este conjunto de dados contém tanto objetos icônicos quanto imagens em nível de cena.

Treinamento. Usamos SGD como nosso otimizador. O peso SGD o decaimento é 0,0001 e o momentum SGD é 0,9. Para IN-1M, usamos um tamanho de minilote de 256 (N no Algoritmo 1) em 8 GPUs e uma taxa de aprendizagem inicial de 0,03. Treinamos por 200 épocas com a taxa de aprendizagem multiplicada por 0,1 em 120 e 160 épocas [61], levando cerca de 53 horas para treinar o ResNet-50. Para o IG-1B, usamos um tamanho de minilote de 1024 em 64 GPUs e uma taxa de aprendizagem de 0,12, que é exponencialmente reduzida em $0,9\times$ após cada 62,5 mil iterações (64 milhões de imagens). Treinamos para 1,25 milhões de iterações ($\sim 1,4$ épocas de IG-1B), levando ~ 6 dias para o ResNet-50.

4.1. Protocolo de classificação linear

Primeiro, verificamos o nosso método por meio de classificação *linear* em características *congeladas*, seguindo um protocolo comum. Nesta subseção, realizamos um pré-treinamento não supervisionado no IN-1M. Em seguida, congelamos as características e treinamos um classificador linear supervisionado (uma camada totalmente conectada seguida por softmax). Treinamos esse classificador nas características de pooling médio global de um ResNet, por 100 épocas. Relatamos a precisão de classificação 1-crop, top-1 no conjunto de validação ImageNet.

Para este classificador, realizamos uma pesquisa em grade e descobrimos que a taxa de aprendizagem inicial ideal é 30 e a decadência de peso é 0 (relatado de forma semelhante em [56]). Esses hiperparâmetros têm um desempenho consistentemente bom para todas as entradas de ablação apresentadas nesta subseção. Esses valores de hiperparâmetros implicam que as distribuições de características (por exemplo, magnitudes) podem ser substancialmente diferentes daquelas do treinamento supervisionado do ImageNet, uma questão que revisitaremos na Seção 4.2.

Ablação: mecanismos de perda contrastiva. Comparamos os três mecanismos ilustrados na Figura 2. Para nos concentrarmos no efeito dos mecanismos de perda contrastiva, implementamos todos eles na mesma tarefa de pretexto, conforme descrito na Sec. 3.3. Também usamos a mesma forma de InfoNCE como função de perda contrastiva, Eqn.(1). Assim, a comparação é feita exclusivamente entre os três mecanismos.

Os resultados estão na Figura 3. No geral, todos os três mecanismos se beneficiam de um K maior. Uma tendência semelhante foi observada em [61, 56] sob o mecanismo de banco de memória, enquanto aqui mostramos que essa tendência é mais geral e pode ser vista em todos os mecanismos. Esses resultados apoiam a nossa motivação de construir um grande dicionário.

O mecanismo *ponta a ponta* tem um desempenho semelhante ao MoCo quando K é pequeno. No entanto, o tamanho do dicionário é limitado pelo tamanho do mini-lote devido ao requisito ponta a ponta. Aqui, o maior mini-lote que uma máquina de ponta (8 GPUs Volta de 32 GB) pode suportar é 1024. Mais essencialmente, o treinamento com mini-lotes grandes é um problema em aberto [25]: consideramos necessário usar a regra de escala de taxa de aprendizagem linear [25] aqui, sem a qual a precisão cai (em $\sim 2\%$ com um mini-lote de 1024). Mas otimizar com um mini-lote maior é mais difícil [25], e é questionável se a tendência pode ser extrapolada para um K maior, mesmo que a memória seja suficiente.

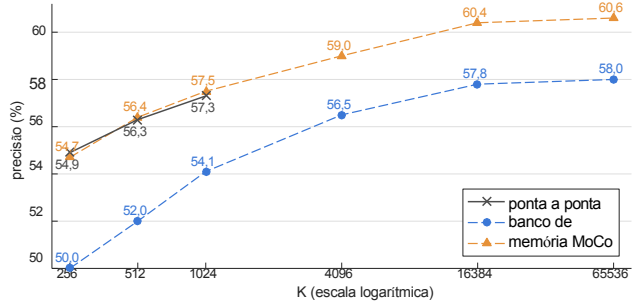


Figura 3. **Comparação de três mecanismos de perda contrastiva** sob o protocolo de classificação linear ImageNet. Adotamos a mesma tarefa pretexto (Sec. 3.3) e variamos apenas o mecanismo de perda contrastiva (Figura 2). O número de negativos é K no banco de memória e MoCo, e é $K-1$ no end-to-end (deslocado em um porque a chave positiva está no mesmo minilote). A rede é ResNet-50.

O mecanismo *do banco de memória* [61] pode suportar um dicionário de tamanho maior. Mas é 2,6% pior do que o MoCo. Isso está de acordo com a nossa hipótese: as chaves no banco de memória são de codificadores muito diferentes ao longo de todo o período anterior e não são consistentes. Observe que o resultado do banco de memória de 58,0% reflete a nossa implementação aprimorada de [61].²

Ablação: momentum. A tabela abaixo mostra a precisão do ResNet-50 com diferentes valores de momentum MoCo (m na Eq. (2)) usados no pré-treinamento ($K = 4096$ aqui):

momento m	0	0,9	0,99	0,999	0,9999
precisão (%)	<i>falha</i>	55,2	57,8	59,0	58,9

O desempenho é razoavelmente bom quando m está entre 0,99 e 0,9999, mostrando que um codificador de chave de progressão lenta (ou seja, com momentum relativamente grande) é benéfico. Quando m é muito pequeno (por exemplo, 0,9), a precisão cai consideravelmente; no extremo *sem momentum* ($m = 0$), a perda de treino oscila e não converge. Estes resultados apoiam a nossa motivação de construir um dicionário consistente.

Comparação com resultados anteriores. Os métodos de aprendizagem não supervisionados anteriores podem diferir substancialmente em tamanhos de modelo. Para uma comparação justa e abrangente, relatamos *a precisão versus o número de parâmetros*³ trade-offs. Além do ResNet-50

(R50) [33], também relatamos as suas variantes que são $2\times$ e $4\times$ mais amplas (mais canais), seguindo [38].⁴ Definimos $K = 65536$ e $m = 0,999$. A Tabela 1 apresenta a comparação.

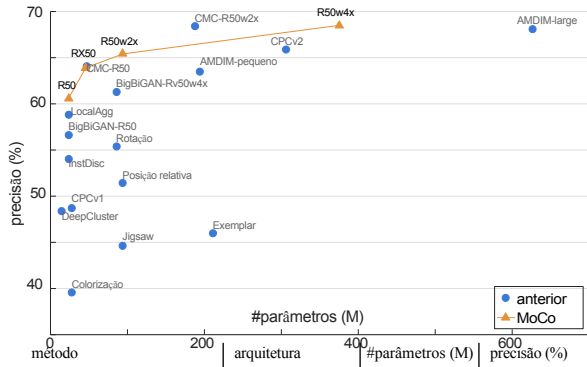
O MoCo com R50 tem um desempenho competitivo e atinge 60,6% de precisão, melhor do que todos os concorrentes de tamanhos de modelo semelhantes ($\sim 24M$). O MoCo se beneficia de modelos maiores e

atinge 68,6% de precisão com **R50w4x**.

Notavelmente, alcançamos resultados competitivos usando um *padrão* ResNet-50 e não requerem projetos de arquitetura específicos, por exemplo,

² Aqui, 58,0% é com InfoNCE e $K=65536$. Reproduzimos 54,3% ao usar NCE e $K=4096$ (o mesmo que [61]), próximo a 54,0% em [61]. ³ Os parâmetros são do *extrator de características*: por exemplo, não contamos os parâmetros do convx se o convx não estiver incluído na classificação linear. parâmetros de convx se convx não estiver incluído na classificação linear.

⁴ Os nossos modelos $w2\times$ e $w4\times$ correspondem aos casos “ $\times 8$ ” e “ $\times 16$ ” em [38], porque o ResNet de tamanho padrão é referido como “ $\times 4$ ” em [38].



método	arquitetura	#parâmetros (M)	precisão (%)
Exemplar [17]	R50w3×	211	46,0 [38]
Posição relativa [13]	R50w2×	94	51,4 [38]
Serra tico-tico [45]	R50w2×	94	44,6 [38]
Rotação [19]	Rv50w4×	86	55,4 [38]
Colorização [64]	R101*	28	39,6 [14]
DeepCluster [3]	VGG [53]	15	48,4 [4]
BigBiGAN [16]	R50	24	56,6
	Rv50w4×	86	61,3
<i>Os métodos baseados na aprendizagem contrastiva seguem:</i>			
InstDisc [61]	R50	24	54,0
LocalAgg [66]	R50 R101*	24	58,8
CPC v1 [46]	R170* mais largo	28	48,7
CPC v2 [35]	R50L+ab	303	65,9
CMC [56]	R50w2× _{L+ab}	47	64,1 [†]
	AMDIM _{pequeno}	188	68,4 [†]
AMDIM [2]	o	194	63,5 [†]
	AMDIM _{grande}	626	68,1 [†]
MoCo	R50	24	60,6
	RX50	46	63,9
	R50w2×	94	65,4
	R50w4×	375	68,6

Tabela 1. Comparação de sob o protocolo de classificação linear no ImageNet. A figura visualiza a tabela. Todos são relatados como pré-treinamento não supervisionado no conjunto de treinamento ImageNet-1M, seguido por classificação linear supervisionada treinada em características congeladas, avaliadas no conjunto de validação. As contagens de parâmetros são as dos extratores de características. Comparamos com reimplementações aprimoradas, se disponíveis (referenciadas após os números).

Notas: R101*/R170* é ResNet-101/170 com o último estágio residual removido [14, 46, 35], e R170 é ampliado [35]; Rv50 é uma rede reversível [23], RX50 é ResNeXt-50-32×8d [62].

[†]: O pré-treinamento usa FastAutoAugment [40] supervisionado por rótulos ImageNet.

entradas patchificadas [46, 35], campos recetivos cuidadosamente adaptados [2] ou a combinação de duas redes [56]. Ao utilizar uma arquitetura que não é personalizada para a tarefa pretexto, é mais fácil transferir características para uma variedade de tarefas visuais e fazer comparações, estudadas na próxima subsecção.

Este artigo centra-se num mecanismo para a aprendizagem contrastiva geral; não exploramos fatores ortogonais (tais como tarefas pré-textuais específicas) que possam melhorar ainda mais a precisão. Por exemplo, o «MoCo v2» [8], uma extensão de uma versão preliminar deste manuscrito, atinge 71,1% de precisão com R50 (um aumento em relação aos 60,6%), com pequenas alterações no aumento de dados e na cabeça de projeção de saída [7]. Acreditamos que este resultado adicional demonstra a generalidade e a robustez da estrutura MoCo.

pré-treinamento	AP ₅₀	AP	AP ₇₅
inicialização aleatória	64,4	37,9	38,6
super. IN-1M	81,4	54,0	59,1
MoCo IN-1M	81,1 (-0,3)	54,6 (+0,6)	59,9 (+0,8)
MoCo IG-1B	81,6 (+0,2)	55,5 (+1,5)	61,2 (+2,1)

(a) Faster R-CNN, R50-dilated-C5

pré-treinamento	AP ₅₀	AP	AP ₇₅
inicialização aleatória	60,2	33,8	33,1
super. IN-1M	81,3	53,5	58,8
MoCo IN-1M	81,5 (+0,2)	55,9 (+2,4)	62,6 (+3,8)
MoCo IG-1B	82,2 (+0,9)	57,2 (+3,7)	63,7 (+4,9)

(b) Faster R-CNN, R50-C4

Tabela 2. Detecção de objetos ajustada no PASCAL VOC trainval07+12. A avaliação é feita no test2007: AP₅₀ (métrica VOC padrão), AP (estilo COCO) e AP₇₅, com média de 5 tentativas. Todos são ajustados para 24 mil iterações (~23 épocas). Entre parênteses estão as diferenças em relação ao pré-treinamento supervisionado ImageNet. Em verde estão as diferenças de pelo menos +0,5 ponto.

pré-treino	R50-dilatado-C5			R50-C4		
	AP ₅₀	AP	AP ₇₅	AP ₅₀	AP	AP ₇₅
ponta a ponta	79,2	52,0	56,6	80,4	54,6	60,3
banco de memória	79,8	52,9	57,9	80,6	54,9	60,6
MoCo	81,1	54,6	59,9	81,5	55,9	62,6

Tabela 3. Comparação de três mecanismos de perda contrastiva na detecção de objetos PASCAL VOC, ajustados em trainval07+12 e avaliados em test2007 (médias de 5 tentativas). Todos os modelos são implementados por nós (Figura 3), pré-treinados em IN-1M e ajustados usando as mesmas configurações da Tabela 2.

4.2. Transferência de características

Um dos principais objetivos da aprendizagem não supervisionada é *aprender características que sejam transferíveis*. O pré-treinamento supervisionado do ImageNet é mais influente quando serve como inicialização para o ajuste fino em tarefas a jusante (por exemplo, [21, 20, 43, 52]). A seguir, comparamos o MoCo com o pré-treinamento supervisionado do ImageNet, transferido para várias tarefas, incluindo PASCAL VOC [18], COCO [42], etc. Como pré-requisitos, discutimos duas questões importantes envolvidas [31]: normalização e cronogramas.

Normalização. Conforme observado na Sec. 4.1, os recursos produzidos pelo pré-treinamento não supervisionado podem ter distribuições diferentes em comparação com o pré-treinamento supervisionado do ImageNet. Mas um sistema para uma tarefa downstream geralmente tem hiperparâmetros (por exemplo, taxas de aprendizagem) selecionados para o pré-treinamento supervisionado. Para aliviar esse problema, adotamos a *normalização de características* durante o ajuste fino: fazemos o ajuste fino com BN que é treinado (e sincronizado entre GPUs [49]), em vez de congelá-lo por uma camada afim [33]. Também usamos BN nas camadas recém-inicializadas (por exemplo, FPN [41]), o que ajuda a calibrar magnitudes.

Realizamos a normalização ao ajustar modelos de pré-treinamento supervisionados e não supervisionados. O MoCo *usa os mesmos hiperparâmetros* que o equivalente supervisionado do ImageNet.

Cronogramas. Se o cronograma de ajuste fino for longo o suficiente, os detetores de treino a partir da inicialização aleatória podem ser bases sólidas e podem corresponder à contraparte supervisionada do ImageNet no COCO [31]. O nosso objetivo é investigar a *transferibilidade*

pré-treinar	AP ₅₀					AP		AP ₇₅	
	RelPos, por [14]	Multitarefa [14]	Jigsaw, por [26]	LocalAgg [66]	MoCo	MoCo		Multitarefa [14]	MoCo
super. IN-1M	74,2	74,2	70,5	74,6	74,4	42,4		44,3	42,7
sem apoio IN-1M	66,8 (-7,4)	70,5 (-3,7)	61,4 (-9,1)	69,1 (-5,5)	74,9 (+0,5)	46,6 (+4,2)		43,9 (-0,4)	50,1 (+7,4)
sem apoio IN-14M	-	-	69,2 (-1,3)	-	75,2 (+0,8)	46,9 (+4,5)		-	50,2 (+7,5)
sem apoio YFCC-100M	-	-	66,6 (-3,9)	-	74,7 (+0,3)	45,9 (+3,5)		-	49,0 (+6,3)
suporte sem apoio IG-1B	-	-	-	-	75,6 (+1,2)	47,6 (+5,2)		-	51,7 (+9,0)

Tabela 4. **Comparação com métodos anteriores de detecção de objetos ajustados no PASCAL VOC** `trainval2007`. A avaliação é feita no `test2007`. As contrapartes supervisionadas do ImageNet são dos respectivos artigos e são relatadas como tendo a mesma estrutura que as respectivas contrapartes de pré-treinamento não supervisionadas. Todas as entradas são baseadas na estrutura C4. Os modelos em [14] são R101 v2 [34] e os outros são R50. O resultado RelPos (posição relativa) [13] é o melhor caso de tarefa única no artigo Multitarefa [14]. O resultado Jigsaw [45] é da implementação baseada em ResNet em [26]. Os nossos resultados são com ajuste fino de 9 mil iterações, com média de 5 tentativas. Entre parênteses estão as diferenças em relação ao pré-treinamento supervisionado ImageNet. Em verde estão as diferenças de pelo menos +0,5 ponto.

idade de características, por isso as nossas experiências são realizadas em horários controlados, por exemplo, os horários $1\times$ (~ 12 épocas) ou $2\times$ [22] para COCO, em contraste com $6\times\sim 9\times$ em [31]. Em conjuntos de dados menores como o VOC, um treino mais longo pode não ser suficiente [31].

No entanto, no nosso ajuste fino, o MoCo *usa o mesmo cronograma* que o equivalente supervisionado do ImageNet, e os resultados da inicialização aleatória são fornecidos como referências.

Em conjunto, o nosso ajuste fino usa a mesma configuração que a contraparte de pré-treinamento supervisionado. Isso pode colocar o MoCo em *desvantagem*. Mesmo assim, o MoCo é competitivo. Isso também torna viável apresentar comparações em vários conjuntos de dados/tarefas, sem pesquisa extra de hiperparâmetros.

4.2.1 Detecção de objetos PASCAL VOC

Configuração. O detetor é o Faster R-CNN [52] com uma estrutura baseada em R50-dilated-C5 ou R50-C4 [32] (detalhes no apêndice), com BN ajustado, implementado em [60]. Ajustamos todas as camadas de ponta a ponta. A escala da imagem é [480, 800] pixels durante o treino e 800 na inferência. A mesma configuração é usada para todas as entradas, incluindo a linha de base de pré-treinamento supervisionado. Avaliamos a métrica VOC padrão de AP₅₀ (ou seja, o limite de IoU é 50%) e as métricas mais rigorosas de AP e AP₇₅ no estilo COCO. A avaliação é feita no conjunto de testes VOC 2007.

Ablação: backbones. A Tabela 2 mostra os resultados ajustados em `trainval07+12` ($\sim 16,5$ mil imagens). Para R50-dilated-C5 (Tabela 2a), o MoCo pré-treinado em IN-1M é comparável ao pré-treinamento supervisionado, e o MoCo pré-treinado em IG-1B o *supera*. Para R50-C4 (Tabela 2b), o MoCo com IN-1M ou IG-1B é *melhor* do que o equivalente supervisionado: até +0,9 AP₅₀, +3,7 AP e +4,9 AP₇₅.

Curiosamente, a precisão da transferência depende da estrutura do detetor. Para a espinha dorsal C4, usada por predefinição nos resultados existentes baseados em ResNet [14, 61, 26, 66], a vantagem do pré-treinamento não supervisionado é maior. A relação entre pré-treinamento e estruturas de detetores foi velada no passado e deve ser um fator a ser considerado.

Ablação: mecanismos de perda contrastiva. Ressaltamos que esses resultados se devem, em parte, ao estabelecimento de bases sólidas de detecção para o aprendizado contrastivo. Para identificar o ganho que é *exclusivamente* contribuído pelo uso do mecanismo MoCo

Na aprendizagem contrastiva, ajustamos os modelos pré-treinados com o mecanismo end-to-end ou banco de memória, ambos implementados por nós (ou seja, os melhores na Figura 3), usando a mesma configuração de ajuste fino que o MoCo.

Esses concorrentes têm um desempenho razoável (Tabela 3). O seu AP e AP₇₅ com o backbone C4 também são superiores aos da contraparte supervisionada ImageNet, cf. Tabela 2b, mas outras métricas são inferiores. Eles são piores que o MoCo em todas as métricas. Isso mostra os benefícios do MoCo. Além disso, como treinar esses concorrentes em dados de maior escala é uma questão em aberto, e eles podem não se beneficiar do IG-1B.

Comparação com resultados anteriores. Seguindo os concorrentes, fazemos o ajuste fino no `trainval2007` (~ 5 mil imagens) usando o backbone C4. A comparação está na Tabela 4.

Para a métrica AP₅₀, *nenhum* método anterior consegue acompanhar a sua respectiva contraparte de pré-treino supervisionado. O MoCo pré-treinado em qualquer um dos IN-1M, IN-14M (ImageNet completo), YFCC-100M [55] e IG-1B pode *superar* a linha de base supervisionada. Grandes ganhos são observados nas métricas mais rigorosas: até +5,2 AP e +9,0 AP₇₅. Esses ganhos são maiores do que os ganhos observados em `trainval07+12` (Tabela 2b).

4.2.2 Detecção e segmentação de objetos COCO

Configuração. O modelo é Mask R-CNN [32] com o backbone FPN [41] ou C4, com BN ajustado, implementado em [60]. A escala da imagem é de [640, 800] pixels durante o treino e é de 800 na inferência. Ajustamos todas as camadas de ponta a ponta. Avaliamos no conjunto `train2017` (~ 118 mil imagens) e avaliamos

no `val2017`. O cronograma é o padrão $1\times$ ou $2\times$ em [22].

Resultados. A Tabela 5 mostra os resultados no COCO com o FPN (Tabela 5a, b) e C4 (Tabela 5c, d). Com o cronograma $1\times$, todos os modelos (incluindo os equivalentes supervisionados pelo ImageNet) estão fortemente subtreinados, conforme indicado pelas diferenças de ~ 2 pontos em relação aos casos do cronograma $2\times$. Com o programa $2\times$, o MoCo é *melhor* do que o seu equivalente supervisionado pelo ImageNet em todas as métricas em ambas as estruturas.

4.2.3 Mais tarefas a jusante

A Tabela 6 mostra mais tarefas a jusante (detalhes de implementação no apêndice). No geral, o MoCo tem um desempenho competitivo

pré-treinamento	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅
inicialização aleatória.	31,0	49,5	33,2	28,5	46,8	30,4	36,7	56,7	40,0	33,7	53,8	35,9
super. IN-1M	38,9	59,6	42,7	35,4	56,5	38,1	40,6	61,3	44,4	36,8	58,1	39,5
MoCo IN-1M	38,5 (-0,4)	58,9 (-0,7)	42,0 (-0,7)	35,1 (-0,3)	55,9 (-0,6)	37,7 (-0,4)	40,8 (+0,2)	61,6 (+0,3)	44,7 (+0,3)	36,9 (+0,1)	58,4 (+0,3)	39,7 (+0,2)
MoCo IG-1B	38,9 (0,0)	59,4 (-0,2)	42,3 (-0,4)	35,4 (0,0)	56,5 (0,0)	37,9 (-0,2)	41,1 (+0,5)	61,8 (+0,5)	45,1 (+0,7)	37,4 (+0,6)	59,1 (+1,0)	40,2 (+0,7)

(a) Mask R-CNN, R50-FPN, programação 1×

pré-treinamento	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅
inicialização aleatória.	26,4	44,0	27,8	29,3	46,9	30,8	35,6	54,6	38,2	31,4	51,5	33,5
super. IN-1M	38,2	58,2	41,2	33,3	54,7	35,2	40,0	59,9	43,1	34,7	56,5	36,9
MoCo IN-1M	38,5 (+0,3)	58,3 (+0,1)	41,6 (+0,4)	33,6 (+0,3)	54,8 (+0,1)	35,6 (+0,4)	40,7 (+0,7)	60,5 (+0,6)	44,1 (+1,0)	35,4 (+0,7)	57,3 (+0,8)	37,6 (+0,7)
MoCo IG-1B	39,1 (+0,9)	58,7 (+0,5)	42,2 (+1,0)	34,1 (+0,8)	55,4 (+0,7)	36,4 (+1,2)	41,1 (+1,1)	60,7 (+0,8)	44,8 (+1,7)	35,6 (+0,9)	57,4 (+0,9)	38,1 (+1,2)

(b) Mask R-CNN, R50-FPN, programação 2×

(c) Mask R-CNN, R50-C4, programação 1×

(d) Mask R-CNN, R50-C4, programação 2×

Tabela 5. **Deteção de objetos e segmentação de instâncias ajustadas no COCO:** AP da caixa delimitadora (AP^{bb}) e AP da máscara (AP^{mk}) avaliadas no val2017. Entre parênteses estão as diferenças em relação à contraparte de pré-treinamento supervisionado do ImageNet. Em verde estão as diferenças de pelo menos +0,5 ponto.

pré-treinamento	Deteção de pontos-chave COCO		
	AP ^{kp}	AP ^{kp} ₅₀	AP ^{kp} ₇₅
inicialização aleatória.	65,9	86,5	71,7
super. IN-1M	65,8	86,9	71,9
MoCo IN-1M	66,8 (+1,0)	87,4 (+0,5)	72,5 (+0,6)
MoCo IG-1B	66,9 (+1,0)	87,8 (+0,9)	73,0 (+1,1)

Estimativa de pose densa COCO

pré-treinar	AP ^{dp}	AP ^{dp} ₅₀	AP ^{dp} ₇₅
inicialização aleatória.	39,4	78,5	35,1
super. IN-1M	48,3	85,6	50,6
MoCo IN-1M	50,1 (+1,8)	86,8 (+1,2)	53,9 (+3,3)
MoCo IG-1B	50,6 (+1,2)	87,0 (+0,2)	54,5 (+3,7)

Segmentação de instâncias LVIS v0.5

pré-treinamento	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅
inicialização aleatória.	22,5	34,8	23,8
super. IN-1M†	24,4	37,8	25,8
MoCo IN-1M	24,1 (-0,3)	37,4 (-0,4)	25,5 (-0,3)
MoCo IG-1B	24,9 (+0,5)	38,2 (+0,4)	26,4 (+0,6)

Instância de paisagens urbanas seg.

pré-treinamento	AP ^{mk}	AP ^{mk} ₅₀	Cityscapes	VOC
inicialização aleatória	25,4	51,1	65,3	39,5
super. IN-1M	32,9	59,6	74,6	74,4
MoCo IN-1M	32,3 (-0,6)	59,3 (-0,3)	75,3 (+0,7)	72,5 (-1,9)
MoCo IG-1B	32,9 (0,0)	60,3 (+0,7)	75,5 (+0,9)	73,6 (-0,8)

Tabela 6. **MoCo vs. pré-treinamento supervisionado ImageNet, ajustado em várias tarefas.** Para cada tarefa, a mesma arquitetura e programação são usadas para todas as entradas (ver apêndice). Entre parênteses estão as diferenças em relação ao pré-treinamento supervisionado ImageNet. Em verde estão as diferenças de pelo menos +0,5 ponto.

† : esta entrada está com BN congelado, o que melhora os resultados; consulte o texto principal.

com pré-treinamento supervisionado ImageNet:

Deteção de pontos-chave COCO: o pré-treinamento supervisionado não tem nenhuma vantagem clara sobre a inicialização aleatória, enquanto o MoCo tem um desempenho superior em todas as métricas.

Estimativa de pose densa COCO [1]: O MoCo supera substancialmente o pré-treinamento supervisionado, por exemplo, em 3,7 pontos em AP^{dp}, nesta tarefa altamente sensível à localização.

Segmentação de instâncias LVIS v0.5 [27]: esta tarefa tem ~1000 categorias distribuídas de cauda longa. Especificamente no LVIS para a linha de base supervisionada ImageNet, descobrimos que o ajuste fino com BN congelado (24,4 AP^{mk}) é melhor do que o cenário ajustável

BN (detalhes no apêndice). Assim, comparamos o MoCo com a variante de pré-treinamento supervisionado melhor nesta tarefa. O MoCo com IG-1B supera-o em todas as métricas.

Segmentação de instâncias de paisagens urbanas [10]: MoCo com IG-1B está em pé de igualdade com a sua contraparte de pré-treinamento supervisionado em AP^{mk} e é superior em AP^{mk}.

Segmentação semântica: No Cityscapes [10], o MoCo supera o seu equivalente de pré-treinamento supervisionado em até 0,9 pontos. Mas na segmentação semântica VOC, o MoCo é pior em pelo menos 0,8 pontos, um caso negativo que observamos.

Resumo. Em suma, o MoCo pode *superar* o seu equivalente pré-treinado supervisionado ImageNet em 7 tarefas de detecção ou segmentação.⁵ Além disso, o MoCo está em pé de igualdade na segmentação de instâncias Cityscapes e fica para trás na segmentação semântica VOC; mostramos outro caso comparável no iNatu-ralist [57] no apêndice. No geral, *o MoCo reduziu significativamente a diferença entre a aprendizagem de representação não supervisionada e supervisionada em várias tarefas de visão.*

Notavelmente, em todas essas tarefas, o MoCo pré-treinado no IG-1B é consistentemente melhor do que o MoCo pré-treinado no IN-1M. Isso mostra que *o MoCo pode ter um bom desempenho neste conjunto de dados em grande escala e relativamente não curado.* Isso representa um para a aprendizagem não supervisionada *no mundo real.*

5. Discussão e conclusão

O nosso método mostrou resultados positivos de aprendizagem não supervisionada em várias tarefas e conjuntos de dados de visão computacional. Algumas questões em aberto merecem ser discutidas. A melhoria do MoCo de IN-1M para IG-1B é consistentemente perceptível, mas relativamente pequena, sugerindo que os dados em maior escala podem não estar a ser totalmente explorados. Esperamos que uma tarefa de pretexto avançada melhore isso. Além da simples tarefa de discriminação de instâncias [61], é possível adotar o MoCo para tarefas de pretexto como auto-codificação mascarada, por exemplo, em linguagem [12] e em visão [46]. Esperamos que o MoCo seja útil em outras tarefas de pretexto que envolvam aprendizagem contrastiva.

⁵ Nomeadamente, detecção de objetos em VOC/COCO, segmentação de instâncias em COCO/LVIS, detecção de pontos-chave em COCO, pose densa em COCO e segmentação semântica em Cityscapes.

A. Apêndice

A.1. Implementação: Backbones de detecção de objetos

As estruturas R50-dilated-C5 e R50-C4 são semelhantes às disponíveis no Detectron2 [60]: (i) *R50-dilated-C5*: a estrutura inclui o estágio ResNet conv₅ com uma dilatação de 2 e stride 1, seguido por uma convolução 3×3 (com BN) que reduz a dimensão para 512. A cabeça de previsão da caixa é composta por duas camadas ocultas totalmente conectadas. (ii) *R50-C4*: a estrutura termina com o estágio conv₄, e a cabeça de previsão da caixa é composta pelo estágio conv₅ (incluindo pooling global) seguido por uma camada BN.

A.2. Implementação: detecção de pontos-chave COCO

Utilizamos Mask R-CNN (versão keypoint) com R50-FPN, implementado em [60], ajustado no COCO train₂₀₁₇ e avaliado no val₂₀₁₇. A programação é 2×.

A.3. Implementação: estimativa de pose densa COCO

Utilizamos o DensePose R-CNN [1] com R50-FPN, implementado em [60], ajustado no COCO train₂₀₁₇ e avaliado em val₂₀₁₇. O cronograma é “s1×”.

A.4. Implementação: segmentação de instâncias LVIS

Utilizamos Mask R-CNN com R50-FPN, ajustado em LVIS [27] treinado em v0.5 e avaliado em val v0.5. Seguimos a linha de base em [27] (arXiv v3 Apêndice B).

O LVIS é um novo conjunto de dados e os designs de modelos nele devem ser explorados. A tabela a seguir inclui as ablações relevantes (todas são médias de 5 tentativas):

pré-treino	BN	1× programação			2× programação		
		AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅
super. IN-1M	congelado	24,1	37,3	25,4	24,4	37,8	25,8
super. IN-1M	ajustado	23,5	36,6	24,8	23,2	36,0	24,4
MoCo IN-1M	ajustado	23,2	36,0	24,7	24,1	37,4	25,5
MoCo IG-1B	ajustado	24,3	37,4	25,9	24,9	38,2	26,4

Uma linha de base de pré-treinamento supervisionado, ajustada de ponta a ponta, mas com BN congelado, tem 24,4 AP^{mk}. Mas ajustar o BN nesta linha de base leva a resultados piores e sobreajuste (isso é diferente do COCO/VOC, onde ajustar o BN proporciona uma precisão melhor ou comparável). O MoCo tem 24,1 AP^{mk} com IN-1M e 24,9 AP^{mk} com IG-1B, ambos superando o pré-treinamento supervisionado na mesma configuração ajustável de BN. Nas melhores configurações individuais, o MoCo ainda pode superar o caso de pré-treinamento supervisionado (24,9 contra 24,4, conforme relatado na Tabela 6 na Seção 4.2).

A.5. Implementação: segmentação semântica

Usamos uma estrutura baseada em FCN [43]. A espinha dorsal consiste nas camadas convolucionais em R50, e as convoluções 3×3 nos blocos conv₅ têm dilatação 2 e stride 1. Isso é seguido por duas convoluções 3×3 extras de 256 canais,

com BN e ReLU e, em seguida, uma convolução 1×1 para classificação por pixel. O stride total é 16 (FCN-16s [43]). Definimos dilatação = 6 nas duas convoluções 3×3 adicionais, seguindo o design de grande campo de visão em [6].

O treino é feito com dimensionamento aleatório (por uma proporção entre [0,5 e 2,0]), recorte e inversão horizontal. O tamanho do recorte é 513 no VOC e 769 no Cityscapes [6]. A inferência é realizada no tamanho original da imagem. Treinamos com tamanho de mini-lote 16 e decaimento de peso 0,0001. A taxa de aprendizagem é 0,003 no VOC e 0,01 no Cityscapes (multiplicada por 0,1 no 70º e 90º percentil do treinamento). Para o VOC, treinamos no conjunto train_{aug2012} (aumentado por [30], 10582 imagens) por 30 mil iterações e avaliamos no val₂₀₁₂. Para o Cityscapes, treinamos no conjunto train_{fine} (2975 imagens) por 90 mil iterações e avaliamos no conjunto val. Os resultados são relatados como médias de 5 tentativas.

A.6. Classificação detalhada do iNaturalist

Além das experiências de detecção/segmentação descritas no artigo principal, estudamos a classificação detalhada no conjunto de dados iNaturalist 2018 [57]. Ajustamos os modelos pré-treinados de ponta a ponta no conjunto de treino (~437 mil imagens, 8142 classes) e avaliamos no conjunto de validação. O treino segue a implementação típica do ResNet no PyTorch com 100 épocas. O ajuste fino tem uma taxa de aprendizagem de 0,025 (vs. 0,1 do zero) reduzida em 10 no 70.º e 90.º percentil do treino. O seguinte é o resultado R50:

pré-treinamento	inicialização aleatória	super. IN-1M	MoCo IN-1M	MoCo IG-1B
precisão (%)	61,8	66,1	65,6	65,8

O MoCo é cerca de 4% melhor do que o treino a partir de inicialização aleatória e é bastante comparável ao seu equivalente supervisionado pelo ImageNet. Isso mostra mais uma vez que o pré-treino não supervisionado do MoCo é competitivo.

A.7. Ajustes finos no ImageNet

A classificação linear em características congeladas (Sec. 4.1) é um protocolo comum de avaliação de métodos de pré-treinamento não supervisionados. No entanto, na prática, é mais comum ajustar as características de ponta a ponta em uma tarefa downstream. Para completar, a tabela a seguir apresenta os resultados do ajuste fino de ponta a ponta para a classificação ImageNet de 1000 classes, em comparação com o treinamento do zero (o ajuste fino usa uma taxa de aprendizagem inicial de 0,03, contra 0,1 do zero):

pré-treinamento	inicialização aleatória	MoCo IG-1B
precisão (%)	76,5	77,3

Como aqui o ImageNet é a tarefa a jusante, o caso do MoCo pré-treinado no IN-1M não representa um cenário real (para referência, informamos que a sua precisão é de 77,0% após o ajuste fino). Mas o pré-treinamento não supervisionado no conjunto de dados *separado* e não rotulado do IG-1B representa um cenário típico: neste caso, o MoCo melhora em 0,8%.

pré-treinar	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅	AP ^{bb}	AP ^{bb} ₅₀	AP ^{bb} ₇₅	AP ^{mk}	AP ^{mk} ₅₀	AP ^{mk} ₇₅
inicialização aleatória.	36,7	56,7	40,0	33,7	53,8	35,9	41,4	61,9	45,1	37,6	59,1	40,3
super. IN-1M	40,6	61,3	44,4	36,8	58,1	39,5	41,9	62,5	45,6	38,0	59,6	40,8
MoCo IN-1M	40,8 (+0,2)	61,6 (+0,3)	44,7 (+0,3)	36,9 (+0,1)	58,4 (+0,3)	39,7 (+0,2)	42,3 (+0,4)	62,7 (+0,2)	46,2 (+0,6)	38,3 (+0,3)	60,1 (+0,5)	41,2 (+0,4)
MoCo IG-1B	41,1 (+0,5)	61,8 (+0,5)	45,1 (+0,7)	37,4 (+0,6)	59,1 (+1,0)	40,2 (+0,7)	42,8 (+0,9)	63,2 (+0,7)	47,0 (+1,4)	38,7 (+0,7)	60,5 (+0,9)	41,3 (+0,5)

(a) Máscara R-CNN, R50-FPN, programação 2×

(b) Máscara R-CNN, R50-FPN, programação 6×

Tabela A.1. Detecção de objetos e segmentação de instâncias ajustadas no COCO: **programação 2×** vs. **6×**. Entre parênteses estão as diferenças em relação ao pré-treinamento supervisionado do ImageNet. Em verde estão as diferenças de pelo menos +0,5 ponto.

A.8. Ajustes mais longos no COCO

Na Tabela 5, relatamos os resultados dos cronogramas 1×

(~12 épocas) e 2×

no COCO. Esses cronogramas foram herdados do artigo original do Mask R-CNN [32], que poderia ser subótimo, dado o avanço posterior na área. Na Tabela A.1, complementamos os resultados de um cronograma 6×

(~72 épocas)

[31] e comparamos com os da programação 2×

Observamos que: (i) o ajuste fino com pré-treinamento supervisionado pelo ImageNet

ainda apresenta melhorias (41,9 AP^{bb}); (ii) o treino a partir do zero alcança em grande parte (41,4 AP^{bb}); (iii) o

As contrapartes MoCo melhoram ainda mais (por exemplo, para 42,8 AP^{bb}) e apresentam diferenças maiores (por exemplo, +0,9 AP^{bb} com 6×

contra +0,5 AP^{bb} com 2×

). A Tabela A.1 e a Tabela 5 sugerem que os recursos pré-treinados MoCo pré-treinados podem ter vantagens *maiores* do que as características supervisionadas pelo ImageNet quando ajustadas *por mais tempo*.

A.9. Ablação em Shuffling BN

A Figura A.1 apresenta as curvas de treino do MoCo com ou sem BN aleatório: a remoção do BN aleatório mostra um claro sobreajuste à tarefa pretexto: a precisão do treino da tarefa pretexto (curva tracejada) aumenta rapidamente para >99,9%, e a precisão da classificação de validação baseada em kNN (curva sólida) cai rapidamente. Isso é observado tanto para as variantes MoCo quanto para as variantes end-to-end; a variante do banco de memória implicitamente tem estatísticas diferentes para q e k , portanto evita esse problema.

Essas experiências sugerem que, sem o BN embaralhado, as estatísticas do sub lote podem servir como uma “assinatura” para indicar em qual sub lote a chave positiva está. O BN embaralhado pode remover essa assinatura e evitar tal trapaça.

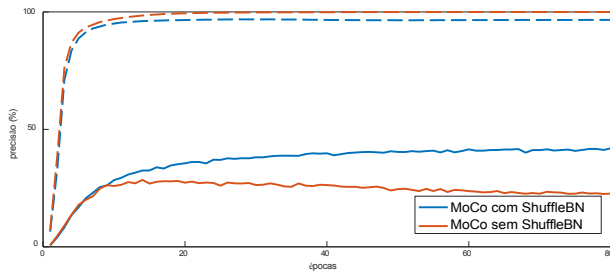


Figura A.1. **Ablação do Shuffling BN.** *Traço:* curva de treino da tarefa pretexto, plotada como a precisão da pesquisa no dicionário (K+1)-way. *Sólido:* curva de validação de um monitor baseado em kNN [61] (não um classificador linear) na precisão da classificação ImageNet. Este gráfico mostra os primeiros 80 épocas de treino: treinar por mais tempo sem o Shuffling BN causa mais sobreajuste.

Referências

- [1] Riza Alp Gu'ler, Natalia Neverova e Iasonas Kokkinos. DensePose: Estimativa densa da postura humana em ambiente natural. Em *CVPR*, 2018.
- [2] Philip Bachman, R Devon Hjelm e William Buchwalter. Aprendendo representações através da maximização da informação mútua entre visões. *arXiv:1906.00910*, 2019.
- [3] Mathilde Caron, Piotr Bojanowski, Armand Joulin e Matthijs Douze. Agrupamento profundo para aprendizagem não supervisionada de características visuais. Em *ECCV*, 2018.
- [4] Mathilde Caron, Piotr Bojanowski, Julien Mairal e Armand Joulin. Pré-treinamento não supervisionado de características de imagem em dados não curados. Em *ICCV*, 2019.
- [5] Ken Chatfield, Victor Lempitsky, Andrea Vedaldi e Andrew Zisserman. O diabo está nos detalhes: uma avaliação de métodos recentes de codificação de características. Em *BMVC*, 2011.
- [6] Liang-Chieh Chen, George Papandreou, Iasonas Kokkinos, Kevin Murphy e Alan L Yuille. DeepLab: segmentação semântica de imagens com redes convolucionais profundas, convolução atrous e CRFs totalmente conectadas. *TPAMI*, 2017.
- [7] Ting Chen, Simon Kornblith, Mohammad Norouzi e Geoffrey Hinton. Uma estrutura simples para aprendizagem contrastiva de representações visuais. *arXiv:2002.05709*, 2020.
- [8] Xinlei Chen, Haoqi Fan, Ross Girshick e Kaiming He. Linhas de base aprimoradas com aprendizagem contrastiva de momentum. *arXiv:2003.04297*, 2020.
- [9] Adam Coates e Andrew Ng. A importância da codificação versus o treino com codificação esparsa e quantização vetorial. Em *ICML*, 2011.
- [10] Marius Cordts, Mohamed Omran, Sebastian Ramos, Timo Rehfeld, Markus Enzweiler, Rodrigo Benenson, Uwe Franke, Stefan Roth e Bernt Schiele. O conjunto de dados Cityscapes para compreensão semântica de cenas urbanas. Em *CVPR*, 2016.
- [11] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li e Li Fei-Fei. ImageNet: uma base de dados hierárquica de imagens em grande escala. Em *CVPR*, 2009.
- [12] Jacob Devlin, Ming-Wei Chang, Kenton Lee e Kristina Toutanova. BERT: Pré-treinamento de transformadores trans es bidirecionais profundos para compreensão da linguagem. Em *NAACL*, 2019.
- [13] Carl Doersch, Abhinav Gupta e Alexei A Efros. Aprendizagem de representação visual não supervisionada por previsão de contexto. Em *ICCV*, 2015.
- [14] Carl Doersch e Andrew Zisserman. Aprendizagem visual supervisionada multitarefa auto . Em *ICCV*, 2017.
- [15] Jeff Donahue, Philipp Kra'henbu'hl e Trevor Darrell. Aprendizagem de características adversárias. Em *ICLR*, 2017.

- [16] Jeff Donahue e Karen Simonyan. Aprendizagem de representação adversária em grande escala com redes neurais convolucionais. *arXiv:1907.02544*, 2019.
- [17] Alexey Dosovitskiy, Jost Tobias Springenberg, Martin Ried-miller e Thomas Brox. Aprendizagem discriminativa não supervisionada de características com redes neurais convolucionais. Em *NeurIPS*, 2014.
- [18] Mark Everingham, Luc Van Gool, Christopher KI Williams, John Winn e Andrew Zisserman. O Desafio Pascal Visual Object Classes (VOC). *IJCV*, 2010.
- [19] Spyros Gidaris, Praveer Singh e Nikos Komodakis. Aprendizagem de representação não supervisionada através da previsão de rotações de imagens. Em *ICLR*, 2018.
- [20] Ross Girshick. Fast R-CNN. Em *ICCV*, 2015.
- [21] Ross Girshick, Jeff Donahue, Trevor Darrell e Jitendra Malik. Hierarquias de características ricas para detecção precisa de objetos e segmentação semântica. Em *CVPR*, 2014.
- [22] Ross Girshick, Ilija Radosavovic, Georgia Gkioxari, Piotr Dollár e Kaiming He. Detectron, 2018.
- [23] Aidan N Gomez, Mengye Ren, Raquel Urtasun e Roger B Grosse. A rede residual reversível: retropropagação de redes neurais sem armazenamento de ativações. Em *NeurIPS*, 2017.
- [24] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville e Yoshua Bengio. Redes adversárias generativas. Em *NeurIPS*, 2014.
- [25] Priya Goyal, Piotr Dollár, Ross Girshick, Pieter Noordhuis, Lukasz Wesolowski, Aapo Kyrola, Andrew Tulloch, Yangqing Jia e Kaiming He. SGD preciso e de grande minilote: Treinamento do ImageNet em 1 hora. *arXiv:1706.02677*, 2017.
- [26] Priya Goyal, Dhruv Mahajan, Abhinav Gupta e Ishan Misra. Dimensionamento e benchmarking da aprendizagem de representação visual auto-supervisionada. Em *ICCV*, 2019.
- [27] Agrim Gupta, Piotr Dollár e Ross Girshick. LVIS: Um conjunto de dados para segmentação de instâncias de vocabulário extenso. Em *CVPR*, 2019.
- [28] Michael Gutmann e Aapo Hyvärinen. Estimativa contrastiva de ruído: um novo princípio de estimativa para modelos estatísticos não normalizados. Em *AISTATS*, 2010.
- [29] Raia Hadsell, Sumit Chopra e Yann LeCun. Redução da dimensionalidade através da aprendizagem de um mapeamento invariante. Em *CVPR*, 2006.
- [30] Bharath Hariharan, Pablo Arbeláez, Lubomir Bourdev, Subhransu Maji e Jitendra Malik. Contornos semânticos a partir de detectores inversos. Em *ICCV*, 2011.
- [31] Kaiming He, Ross Girshick e Piotr Dollár. Repensando o pré-treinamento do ImageNet. Em *ICCV*, 2019.
- [32] Kaiming He, Georgia Gkioxari, Piotr Dollár e Ross Girshick. Mask R-CNN. Em *ICCV*, 2017.
- [33] Kaiming He, Xiangyu Zhang, Shaoqing Ren e Jian Sun. Aprendizagem residual profunda para reconhecimento de imagens. Em *CVPR*, 2016.
- [34] Kaiming He, Xiangyu Zhang, Shaoqing Ren e Jian Sun. Mapeamentos de identidade em redes residuais profundas. Em *ECCV*, 2016.
- [35] Olivier J Héroux, Ali Razavi, Carl Doersch, SM Eslami e Aaron van den Oord. Reconhecimento de imagens com eficiência de dados com codificação preditiva contrastiva. *arXiv:1905.09272*, 2019. Versão atualizada acessada em <https://openreview.net/pdf?id=rJerHlrYwH>.
- [36] R Devon Hjelm, Alex Fedorov, Samuel Lavoie-Marchildon, Karan Grewal, Adam Trischler e Yoshua Bengio. Aprendizagem de representações profundas por estimativa e maximização de informação mútua. Em *ICLR*, 2019.
- [37] Sergey Ioffe e Christian Szegedy. Normalização em lote: acelerando o treinamento de redes profundas através da redução da mudança de covariáveis internas. Em *ICML*, 2015.
- [38] Alexander Kolesnikov, Xiaohua Zhai e Lucas Beyer. Revisitando a aprendizagem de representação visual auto-supervisionada. Em *CVPR*, 2019.
- [39] Yann LeCun, Bernhard Boser, John S Denker, Donnie Henderson, Richard E Howard, Wayne Hubbard e Lawrence D Jackel. Retropropagação aplicada ao reconhecimento de códigos postais manuscritos. *Computação neural*, 1989.
- [40] Sungbin Lim, Ildoo Kim, Taesup Kim, Chiheon Kim e Sungwoong Kim. Fast AutoAugment. *arXiv:1905.00397*, 2019.
- [41] Tsung-Yi Lin, Piotr Dollár, Ross Girshick, Kaiming He, Bharath Hariharan e Serge Belongie. Redes piramidais de características para detecção de objetos. Em *CVPR*, 2017.
- [42] Tsung-Yi Lin, Michael Maire, Serge Belongie, James Hays, Pietro Perona, Deva Ramanan, Piotr Dollár e C Lawrence Zitnick. Microsoft COCO: objetos comuns em contexto. Em *ECCV*, 2014.
- [43] Jonathan Long, Evan Shelhamer e Trevor Darrell. Redes totalmente convolucionais para segmentação semântica. Em *CVPR*, 2015.
- [44] Dhruv Mahajan, Ross Girshick, Vignesh Ramanathan, Kaiming He, Manohar Paluri, Yixuan Li, Ashwin Bharambe e Laurens van der Maaten. Explorando os limites do pré-treinamento supervisionado fracamente e e. Em *ECCV*, 2018.
- [45] Mehdi Noroozi e Paolo Favaro. Aprendizagem não supervisionada de representações visuais através da resolução de quebra-cabeças. Em *ECCV*, 2016.
- [46] Aaron van den Oord, Yazhe Li e Oriol Vinyals. Aprendizagem de representações com codificação preditiva contrastiva. *arXiv:1807.03748*, 2018.
- [47] Deepak Pathak, Ross Girshick, Piotr Dollár, Trevor Darrell e Bharath Hariharan. Aprendizagem de características observando o movimento de objetos. Em *CVPR*, 2017.
- [48] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell e Alexei A Efros. Codificadores de contexto: aprendizagem de características por preenchimento. Em *CVPR*, 2016.
- [49] Chao Peng, Tete Xiao, Zeming Li, Yuning Jiang, Xiangyu Zhang, Kai Jia, Gang Yu e Jian Sun. MegDet: Um grande detetor de objetos por mini-lotes. Em *CVPR*, 2018.
- [50] Alec Radford, Karthik Narasimhan, Tim Salimans e Ilya Sutskever. Melhorando a compreensão da linguagem por meio do pré-treinamento generativo. 2018.
- [51] Alec Radford, Jeffrey Wu, Rewon Child, David Luan, Dario Amodei e Ilya Sutskever. Os modelos de linguagem são aprendizes multitarefas não supervisionados. 2019.
- [52] Shaoqing Ren, Kaiming He, Ross Girshick e Jian Sun. Faster R-CNN: Rumo à detecção de objetos em tempo real com redes de proposta de regiões. Em *NeurIPS*, 2015.

- [53] Karen Simonyan e Andrew Zisserman. Redes convolucionais muito profundas para reconhecimento de imagens em grande escala. Em *ICLR*, 2015.
- [54] Josef Sivic e Andrew Zisserman. Video Google: uma abordagem de recuperação de texto para correspondência de objetos em vídeos. Em *ICCV*, 2003.
- [55] Bart Thomee, David A Shamma, Gerald Friedland, Ben-jamin Elizalde, Karl Ni, Douglas Poland, Damian Borth e Li-Jia Li. YFCC100M: Os novos dados na investigação multimédia. *Communications of the ACM*, 2016.
- [56] Yonglong Tian, Dilip Krishnan e Phillip Isola. Codificação multiview contrastiva. *arXiv:1906.05849*, 2019. Versão atualizada acessada em <https://openreview.net/pdf?id=BkgStySKPB>.
- [57] Grant Van Horn, Oisin Mac Aodha, Yang Song, Yin Cui, Chen Sun, Alex Shepard, Hartwig Adam, Pietro Perona e Serge Belongie. O conjunto de dados de classificação de espécies e detecção de objetos do iNaturalist. Em *CVPR*, 2018.
- [58] Pascal Vincent, Hugo Larochelle, Yoshua Bengio e Pierre-Antoine Manzagol. Extração e composição de características robustas de representação visual com autoencoders de redução de ruído. Em *ICML*, 2008.
- [59] Xiaolong Wang e Abhinav Gupta. Aprendizagem não supervisionada de representações visuais usando vídeos. Em *ICCV*, 2015.
- [60] Yuxin Wu, Alexander Kirillov, Francisco Massa, Wan-Yen Lo e Ross Girshick. Detectron2. <https://github.com/facebookresearch/detectron2>, 2019.
- [61] Zhirong Wu, Yuanjun Xiong, Stella Yu e Dahua Lin. Aprendizagem de características não supervisionada através da discriminação de instâncias não paramétricas. Em *CVPR*, 2018. Versão atualizada acessada em: <https://arxiv.org/abs/1805.01978v1>.
- [62] Saining Xie, Ross Girshick, Piotr Dollár, Zhuowen Tu e Kaiming He. Transformações residuais agregadas para redes neurais profundas. Em *CVPR*, 2017.
- [63] Mang Ye, Xu Zhang, Pong C Yuen e Shih-Fu Chang. Aprendizagem de incorporação não supervisionada por meio de características invariantes e difusas de instâncias de. Em *CVPR*, 2019.
- [64] Richard Zhang, Phillip Isola e Alexei A Efros. Colorização de imagens com colorido. Em *ECCV*, 2016.
- [65] Richard Zhang, Phillip Isola e Alexei A Efros. Autoencoders split-brain: aprendizagem não supervisionada por pre-dição entre canais. Em *CVPR*, 2017.
- [66] Chengxu Zhuang, Alex Lin Zhai e Daniel Yamins. Agregação local para aprendizagem não supervisionada de incorporações visuais. Em *ICCV*, 2019. Resultados adicionais acessados a partir de materiais complementares.